

BE AIML — Machine Learning

Assignment Answers

Units 1 & 2 — Complete Study Guide

Short Answers | Long Answers | Algorithms | Solved Problems

ML Disciplines | Hypothesis Space | FIND-S | Candidate Elimination | ID3 | MLE | MAP

Question	Topic
Q1	Disciplines influencing Machine Learning — with examples
Q2	Choices in Designing a Learning Example — complete framework
Q3	Appropriate & Inappropriate applications of Machine Learning
Q4	Inductive Hypothesis, General-to-Specific ordering, Version Space, Boundaries
Q5	FIND-S Algorithm — step-by-step with EnjoySport example
Q6	List-Then-Eliminate Algorithm — complete description
Q7	Candidate Elimination Algorithm — full trace with 4 training examples
Q8	Entropy & Information Gain — fully solved numerical problem
Q9	MLE & MAP — definitions, proofs, and numerical examples
Q10	Decision Trees for Boolean functions: $A \wedge \neg B$ and $A \vee [B \wedge C]$

Unit 1 — Introduction to Machine Learning

Disciplines | Learning Examples | Applications | Hypothesis Space | FIND-S | Candidate Elimination

Long Answer

Q1. Briefly describe the various disciplines and examples of their influence on Machine Learning.

Machine Learning draws from many disciplines. Each field contributes unique techniques, ideas, and frameworks that together make modern ML possible.

1. Statistics — The most foundational influence on ML.

- Provides tools for estimating probability distributions, hypothesis testing, and inference
- Bayesian statistics → Naive Bayes, MAP estimation, probabilistic graphical models
- Maximum Likelihood Estimation (MLE) is directly used for training many models

Example: Linear regression, logistic regression — both rooted in statistical estimation theory. Bias-variance tradeoff and regularization are statistical concepts central to ML.

2. Computer Science and Artificial Intelligence

- Search algorithms → Hypothesis space search in concept learning (FIND-S, Candidate Elimination)
- Computational complexity → determines which learning algorithms are feasible
- Knowledge representation → features, attributes, logical expressions

Example: Decision trees evolved from AI rule-induction systems. Search heuristics like A* inspired greedy approaches in decision tree building (ID3).

3. Neuroscience / Biology

- The brain's structure of interconnected neurons inspired Artificial Neural Networks (ANNs)
- The Perceptron was directly inspired by McCulloch-Pitts neuron model
- Reinforcement Learning mirrors behavioral conditioning (reward/punishment in animals)

Example: Deep Learning — CNNs mimic the visual cortex's hierarchical feature detection. The brain recognizes edges→shapes→objects; CNNs do the same layer by layer.

4. Mathematics and Linear Algebra

- Calculus → gradient descent for optimizing weights in neural networks
- Linear algebra → matrix operations power all modern deep learning
- Optimization theory → convex optimization for SVM, loss minimization

Example: SVM is a quadratic optimization problem solved using Lagrange multipliers. PCA for dimensionality reduction requires computing eigenvalues of covariance matrix.

5. Psychology and Cognitive Science

- Human learning theories → concept formation, generalization, memory
- Gestalt psychology → inspired clustering and similarity-based learning
- Cognitive models → informed the design of reinforcement learning reward systems

Example: k-NN (k-Nearest Neighbors) mimics human analogical reasoning — "This new fruit looks like the bananas I know, so it's probably also a banana."

6. Information Theory (Claude Shannon)

- Entropy concept → used directly in ID3 decision tree algorithm for attribute selection
- Information gain → measures how much an attribute reduces uncertainty
- Kolmogorov complexity → minimum description length (MDL) principle

Example: ID3 algorithm chooses the attribute with highest information gain at each node. Entropy $H(S) = -\sum [p \log_2(p)]$ is Shannon's entropy applied to classification.

7. Control Theory and Robotics

- Feedback control systems → inspired Q-Learning and Temporal Difference methods
- Adaptive control → adjusting system parameters based on observed outcomes

Example: Robot arm learning to reach a target — uses RL with reward=distance reduced. Q-Learning update rule parallels the Bellman equation from control theory.

Long Answer

Q2. Summarize the choices in designing a Learning Example with a suitable example.

Tom Mitchell identified 5 fundamental choices that must be made when designing any machine learning system. These choices define the problem completely.

Choice 1 — What type of Training Experience?

- **Direct feedback:** Output label given for each specific input example (most common, supervised learning)
- **Indirect feedback:** Only final outcome known — e.g., win/loss at end of chess game (not per move)
- **Teacher control:** Does a teacher provide examples, or does the learner choose what to learn? (Active Learning)
- **Representative sample:** Training data must reflect real distribution — biased training = biased model

Choice 2 — What exactly to learn (Target Function)?

- Define precisely WHAT function $f: X \rightarrow Y$ the system must learn
- For chess: Learn evaluation function $V(\text{board}) \rightarrow \text{numeric score}$
- For email: Learn $f(\text{email_features}) \rightarrow \{\text{Spam}, \text{Not-Spam}\}$
- Can also learn policy, conditional probability, or feature transformation

Choice 3 — How to Represent the Target Function?

- Conjunctions of attribute-value pairs: — concept learning
- Linear functions: $V = w_0 + w_1*f_1 + w_2*f_2 + \dots + w_n*f_n$
- Decision trees: Hierarchical yes/no splits on attributes
- Neural networks: Weighted layers of non-linear transformations
- Instance-based: Store examples; predict using nearest stored neighbor

Choice 4 — What Learning Algorithm?

- Gradient descent: Adjust weights to minimize error iteratively
- Version space search: Candidate Elimination narrows hypothesis space

- Information gain: ID3 builds decision tree by greedy attribute selection
- Backpropagation: Train neural networks using chain rule of calculus

Choice 5 — How to Evaluate the Learned Function?

- Training accuracy vs. test accuracy (generalization performance)
- k-Fold cross-validation for reliable, unbiased performance estimate
- Precision/Recall/F1-score for imbalanced classification tasks

COMPLETE EXAMPLE — Designing a Chess ML System (Mitchell, 1997): Choice 1 — Training Experience: Indirect: outcome (win/loss/draw) at END of each game NOT labeled per-move — must infer which moves were good/bad Choice 2 — Target Function: V : BoardState $\rightarrow$ Real number (positive = good for white, negative = bad) Choice 3 — Representation: $V(b) = w_0 + w_1 \# \text{white_pawns} + w_2 \# \text{black_pawns} + w_3 \# \text{white_queens} + w_4 \# \text{black_queens} + w_5 \# \text{white_rooks} + w_6 \# \text{black_rooks}$ Choice 4 — Learning Algorithm: LMS (Least Mean Squares) gradient descent: $V_{\text{train}}(b_t) = V(b_{t+1})$ [current board value = next board value] $w_i = w_i + \eta (V_{\text{train}}(b) - V(b)) * f_i(b)$ Choice 5 — Evaluation: Win rate vs. baseline chess engine over 1000 games

Answer

Q3. Give 3 applications where ML is APPROPRIATE and 3 where it is INAPPROPRIATE.

ML is APPROPRIATE when: The problem is too complex to hand-code rules, patterns exist in data, rules change over time, or performance improves with experience.

Application	Why ML is Appropriate
1. Email Spam Detection	Patterns in spam evolve constantly; too many rules to code manually; ML adapts automatically to new spam tactics
2. Medical Diagnosis / Image Analysis	Too many subtle patterns in X-rays/MRIs for humans to fully specify; ML finds patterns in millions of scans
3. Speech Recognition	Pronunciation varies by person, accent, noise; impossible to write rules for all variations; learns from examples

ML is INAPPROPRIATE when: The solution can be precisely programmed, data is very scarce, decisions must be fully explainable/auditable, or errors are catastrophic.

Application	Why ML is Inappropriate
1. Simple Calculator	$4+5=9$ is a deterministic rule — no learning needed; correctness must be guaranteed; any error is unacceptable
2. Legal Compliance Checking (Rule-based)	Laws are explicit, precisely stated rules; ML could misinterpret edge cases; requires 100% auditability
3. Safety-Critical Flight Control	Software errors are fatal; system behavior must be formally verified; no room for probabilistic errors

Q4. Define with suitable examples: Inductive Learning Hypothesis | General-to-Specific Ordering | General/Specific Boundary | Version Space | Version Space Representation Theorem

(a) The Inductive Learning Hypothesis:

The inductive learning hypothesis states that any hypothesis found to approximate the target function well over a sufficiently large, representative set of training examples, will also approximate the target function well over other unobserved examples.

"Any hypothesis h found to approximate the target function well over a sufficiently large training set D will also approximate it well over previously unseen examples." – Tom Mitchell

Example: A spam filter trained on 10,000 labeled emails that achieves 98% accuracy on training data should also achieve ~98% on new emails — IF the training emails are representative of future emails (same distribution). Fails when: Training data has selection bias (e.g., trained only on formal emails, tested on casual text messages — distribution mismatch).

(b) General-to-Specific Ordering of Hypotheses:

In a hypothesis space H , hypothesis h_j is MORE GENERAL than h_i ($h_j \geq h_i$) if every example classified as positive by h_i is also classified as positive by h_j . The most general hypothesis accepts everything; the most specific accepts nothing (or exactly one example).

Definition: $h_j \geq h_i$ iff $\{x : h_i(x)=1\} \subseteq \{x : h_j(x)=1\}$ " h_j is at least as general as h_i "

Example (EnjoySport — Sky, Temp, Humidity, Wind, Water, Forecast): h_1 = [very specific — one exact case] h_2 = [more general — any Sunny+Strong day] h_3 = [most general — accepts everything] Ordering: $h_1 \leq h_2 \leq h_3$ (h_1 is more specific than h_2 which is more specific than h_3) This ordering provides a structure for searching the hypothesis space efficiently.

(c) General Boundary (G) and Specific Boundary (S):

The version space $VS(H,D)$ can be compactly represented by two boundaries:

- **S — Specific Boundary:** The set of maximally specific (least general) hypotheses in H that are consistent with ALL training examples. Start: $S = \{\emptyset, \emptyset, \emptyset, \emptyset, \emptyset\}$
- **G — General Boundary:** The set of maximally general (most general) hypotheses in H that are consistent with training examples. Start: $G = \{?, ?, ?, ?, ?\}$

After processing 4 EnjoySport examples: $S = \{ \}$ — specific boundary $G = \{ , \}$ — general boundary Any hypothesis h with $S \leq h \leq G$ is consistent with all training data. These two sets together completely represent the entire version space.

(d) Version Space:

The Version Space $VS(H,D)$ is the subset of all hypotheses in H that are consistent with every training example in D .

$VS(H,D) = \{h \in H \mid h \text{ is consistent with every example in } D\}$ Consistent: $h(x) = c(x)$ for all training examples $(x, c(x)) \in D$

If H has 32 possible hypotheses (e.g., for 2 binary attributes): All 32 hypotheses consistent with 0 training examples $\rightarrow$ full version space After 1 positive example: 16 hypotheses eliminated $\rightarrow$ VS has 16 members After 3 examples: possibly only 2-3 hypotheses remain consistent When VS has 1 member: we have found the unique consistent hypothesis When VS is empty: no hypothesis in H is consistent $\rightarrow$ need richer H

(e) Version Space Representation Theorem:

Version Space Representation Theorem: Let H be a hypothesis space, c be the target concept, and D be a set of training examples. For any hypothesis $h \in H$: $h \in VS(H, D)$ if and only if there exists some $s \in S$ and $g \in G$ such that: $g \geq h \geq s$ i.e., h lies between some S member (below) and some G member (above) in the general-to-specific ordering.

This theorem is powerful because it means we only need to track S and G — two boundary sets — instead of all members of VS. The Candidate Elimination Algorithm uses this theorem.

Algorithm

Q5. Describe the FIND-S Algorithm — Finding a Maximally Specific Hypothesis

FIND-S finds the maximally specific hypothesis that is consistent with all positive training examples. It starts with the most specific possible hypothesis and generalizes it minimally when a positive example is not covered. Negative examples are completely ignored.

Algorithm:

- Step 1: Initialize h to the most specific hypothesis in H : $h \leftarrow \langle \emptyset, \emptyset, \emptyset, \dots, \emptyset \rangle$
- Step 2: For each POSITIVE training example $x = \langle x_1, x_2, \dots, x_n \rangle$:
- For each attribute constraint a_i in h :
- IF $h[i] = \emptyset$ (empty/null): $h[i] \leftarrow x_i$ (set to this value)
- ELSE IF $h[i] \neq x_i$: $h[i] \leftarrow ?$ (generalize to "any")
- ELSE ($h[i] = x_i$): $h[i]$ unchanged (already consistent)
- Step 3: Negative examples: SKIP entirely — FIND-S ignores them
- Step 4: Return hypothesis h

FIND-S Traced on EnjoySport Examples: Attributes: Sky, Airtemp, Humidity, Wind, Water, Forecast Init: $h = \langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$ Ex1(+): $h = [\text{all } \emptyset \text{ replaced}]$ Ex2(+): Compare: Humidity=Normal vs High $\rightarrow$ differ $\rightarrow$ generalize to ? $h = \text{Ex3}(-)$: $\rightarrow$ IGNORED Ex4(+): Compare: Water=Warm vs Cold $\rightarrow$ differ $\rightarrow$ generalize to ? Forecast=Same vs Change $\rightarrow$ differ $\rightarrow$ generalize to ? $h = \text{FINAL HYPOTHESIS}$: $h = \text{Limitations of FIND-S}$: 1. Returns only ONE hypothesis — does not track alternatives 2. Ignores negative examples — may not detect inconsistencies 3. Biased toward most specific — may miss other consistent hypotheses 4. If training data has errors, FIND-S may produce incorrect results

Algorithm

Q6. Describe the List-Then-Eliminate Algorithm

The **List-Then-Eliminate** algorithm is the brute-force method for finding the version space. It explicitly lists ALL hypotheses in H and eliminates those inconsistent with each training example.

Algorithm:

- Step 1: VersionSpace $\leftarrow$ list of every hypothesis in H (enumerate all of H)
- Step 2: For each training example $\langle x, c(x) \rangle$:

- Remove from VersionSpace every hypothesis h that predicts $h(x) \neq c(x)$
- i.e., keep only consistent hypotheses
- Step 3: Return VersionSpace

$$\text{VersionSpace} = \{h \in H \mid \text{for all } (x, c(x)) \text{ in } D: h(x) = c(x)\}$$

Simple Example — $H = \{h_1, h_2, h_3, h_4, h_5\}$ (5 hypotheses): Start: $VS = \{h_1, h_2, h_3, h_4, h_5\}$ Example 1 (+): h_3 predicts NO for this $\rightarrow$ remove h_3 $VS = \{h_1, h_2, h_4, h_5\}$ Example 2 (-): h_1 predicts YES for this $\rightarrow$ remove h_1 $VS = \{h_2, h_4, h_5\}$ Example 3 (+): h_5 predicts NO for this $\rightarrow$ remove h_5 $VS = \{h_2, h_4\}$ Final: $VS = \{h_2, h_4\}$ — both hypotheses are consistent with all examples CRITICAL PROBLEM: If $|H|$ is large (even moderate attributes give millions of hypotheses), storing all of H is computationally infeasible. Solution: Candidate Elimination uses S and G boundaries instead.

Comparison with FIND-S and Candidate Elimination:

Algorithm	Tracks	Uses	Returns	Issue
FIND-S	Only most specific consistent h	Negative examples	Single hypothesis	Ignores other consistent h
List-Then-Eliminate	Entire VS	All examples	All consistent h	Too slow for large H
Candidate Elimination	S and G boundaries	All examples	Complete VS compactly	Practical and complete

Algorithm

Q7. Describe Candidate Elimination Algorithm and Trace it on EnjoySport examples

The **Candidate Elimination Algorithm** efficiently represents the complete version space using just two boundary sets: S (most specific consistent hypotheses) and G (most general consistent hypotheses). It processes both positive and negative examples.

Algorithm:

- Initialize: $S \leftarrow \{\text{most specific hypothesis } \langle \emptyset, \emptyset, \dots, \emptyset \rangle\}$
- Initialize: $G \leftarrow \{\text{most general hypothesis } \langle ?, \dots, ? \rangle\}$
- FOR each training example $d = \langle x, c(x) \rangle$:
 - IF d is POSITIVE ($c(x) = 1$):
 - Remove from G any h inconsistent with d
 - For each $s \in S$ that does NOT cover d :
 - Generalize s minimally so it covers d
 - Remove new s' if less general than some $h \in G$
 - Remove old members of S more specific than s'
 - IF d is NEGATIVE ($c(x) = 0$):
 - Remove from S any h inconsistent with d (should cover no negatives)
 - For each $g \in G$ that COVERS d (wrongly classifies it +):
 - Specialize g minimally so it no longer covers d
 - Remove new g' if more specific than some $h \in S$
 - Remove old members of G more general than g'
- RETURN S and G

Training Data:

Ex	Sky	Airtemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	YES (+)
2	Sunny	Warm	High	Strong	Warm	Same	YES (+)
3	Rainy	Cold	High	Strong	Warm	Change	NO (-)
4	Sunny	Warm	High	Strong	Cold	Change	YES (+)

CANDIDATE ELIMINATION — Complete Trace: Initialize: $S_0 = \{ \langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle \}$ (empty — most specific)
 $G_0 = \{ \}$ (any — most general)

Example 1 (+): → YES Positive example: Generalize S to cover it. $S_0 = \{ \langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle \}$ does not cover Ex1 → generalize: Replace each $\emptyset$ with the actual value from Ex1 $S_1 = \{ \}$ G: Already covers Ex1 → $G_1 = \{ \}$ [unchanged]

Example 2 (+): → YES Positive example: S_1 must cover Ex2. Current S_1 = Humidity: Normal $\neq$ High → generalize to ? All other attributes match $S_2 = \{ \}$ G: Already covers Ex2 → $G_2 = \{ \}$ [unchanged]

Example 3 (-): → NO Negative example: Must specialize G to EXCLUDE this example. Current G member covers Ex3 (predicts YES) → must specialize. Create minimal specializations of that exclude Ex3: Change one attribute to NOT match Ex3: $g_1 = [\text{Sky}=\text{Sunny}, \text{Ex3 has Rainy} \rightarrow \text{excludes Ex3}]$ $g_2 = [\text{Temp}=\text{Warm}, \text{Ex3 has Cold} \rightarrow \text{excludes Ex3}]$ $g_3 = [\text{Humidity}=\text{Normal}, \text{Ex3 has High} \rightarrow \text{excludes Ex3}]$ $g_4 = [\text{Water}=\text{Cool}, \text{Ex3 has Warm} \rightarrow \text{excludes Ex3}]$ $g_5 = [\text{Forecast}=\text{Same}, \text{Ex3 has Change} \rightarrow \text{excludes Ex3}]$ Check each against $S_2 = \{ \}$: $g_3 = \{ \}$: S_2 has ?=any → not more specific than S_2 → KEEP $g_4 = \{ \}$: S_2 has Water=Warm $\neq$ Cool → not more specific → REMOVE Keeping only g members that are consistent with positive examples: $G_3 = \{ , , \}$ $S_3 = \{ \}$ [S unchanged — correct on negatives]

Example 4 (+): → YES Positive example: S_3 must cover Ex4. Current S_3 = Water: Warm $\neq$ Cold → generalize to ? Forecast: Same $\neq$ Change → generalize to ? Tentative new s = Check: Is new s consistent with G_3 ? $\geq_g$ → YES, consistent $S_4 = \{ \}$ Update G: Remove members not consistent with Ex4: says Forecast=Same, but Ex4 has Change → REMOVE $G_4 = \{ , \}$

FINAL VERSION SPACE: $S = \{ \}$ $G = \{ , \}$ Classification of $H' = \{ \vee \}$: $h_1 = \{ \}$: Check vs S: $S = h_1$ has Temp=Cold, S requires Warm → h_1 NOT more general than S → NOT in VS $h_2 = \{ \}$: Check vs G: Is $h_2 \leq_g$ some g in G? is more general than → YES Check vs S: Is $h_2 \geq_g$ S=? h_2 has Temp=? (any), S has Temp=Warm (specific) → h_2 more general → YES h_2 IS between S and G → h_2 IS in VS (consistent with all training examples)

Unit 2 — Decision Trees, Entropy, MLE, MAP, Decision Trees

Information Gain | ID3 Algorithm | MLE Proof | MAP Proof | Boolean Decision Trees

Problem

Q8. Entropy & Information Gain — Fully Solved Numerical Problem

Training set: 6 instances with attributes A1 and A2. Find: (a) Entropy of collection, (b) Information Gain of A2.

Instance	Classification	A1	A2
1	+	T	T
2	+	T	T
3	-	T	F
4	+	F	F
5	-	F	T
6	-	F	T

SOLUTION Data Summary: Total = 6 examples Positive (+): Instances 1, 2, 4 → $n_+ = 3$ Negative (-): Instances 3, 5, 6 → $n_- = 3$

PART (a) — ENTROPY of the entire collection: Formula: $H(S) = -\sum [p(i) \times \log(p(i))]$ $p(+) = 3/6 = 0.5$ $p(-) = 3/6 = 0.5$ $H(S) = -(0.5 \times \log(0.5)) - (0.5 \times \log(0.5)) = -(0.5 \times (-1)) - (0.5 \times (-1)) = 0.5 + 0.5 = 1.0$ bit Interpretation: $H=1.0$ is maximum entropy — perfectly mixed (50% + and 50% -). We have maximum uncertainty about the class of any example.

PART (b) — INFORMATION GAIN of A2: Formula: $IG(S, A2) = H(S) - \sum_v [|S_v|/|S| \times H(S_v)]$ Step 1: Split examples by value of A2: A2 = T: Instances 1(+), 2(+), 5(-), 6(-) → 4 examples: 2+, 2- A2 = F: Instances 3(-), 4(+) → 2 examples: 1+, 1- Step 2: Compute entropy of each subset: $H(A2=T)$: $p(+) = 2/4 = 0.5$, $p(-) = 2/4 = 0.5$ $H = -(0.5 \times \log(0.5)) - (0.5 \times \log(0.5)) = 0.5 + 0.5 = 1.0$ bit $H(A2=F)$: $p(+) = 1/2 = 0.5$, $p(-) = 1/2 = 0.5$ $H = -(0.5 \times \log(0.5)) - (0.5 \times \log(0.5)) = 0.5 + 0.5 = 1.0$ bit Step 3: Compute weighted entropy after split: Weighted = $(4/6) \times H(T) + (2/6) \times H(F) = (0.667 \times 1.0) + (0.333 \times 1.0) = 0.667 + 0.333 = 1.0$ bit Step 4: Information Gain: $IG(S, A2) = H(S) - \text{Weighted Entropy} = 1.0 - 1.0 = 0.0$ bits ANSWERS: (a) Entropy $H(S) = 1.0$ bit (maximum — completely mixed classes) (b) $IG(S, A2) = 0.0$ bits (A2 provides NO information for classification) Conclusion: Attribute A2 is completely useless for building a decision tree. Both subsets (A2=T and A2=F) are equally impure as the original set. A useful attribute would produce $IG > 0$ (at least one pure or purer subset).

Long Answer

Q9. Define, State and Prove: (a) MLE Algorithm (b) MAP Algorithm

(a) Maximum Likelihood Estimation (MLE):

Definition: MLE finds the hypothesis h that maximizes the probability of observing the training data D given hypothesis h . It is a purely frequentist approach that ignores prior knowledge.

$h_{MLE} = \text{argmax}_h P(D | h)$ For i.i.d. examples: $h_{MLE} = \text{argmax}_h \prod_i P(d_i | h)$
Log form (easier): $h_{MLE} = \text{argmax}_h \sum_i \log P(d_i | h)$

PROOF — MLE for Binomial (Coin Flipping):

Let $\theta = P(\text{Head})$. Observe data D with n_H heads and n_T tails. Find θ that maximizes $P(D|\theta)$.

Likelihood function: $L(\theta) = P(D|\theta) = \theta^{n_H} \times (1-\theta)^{n_T}$ Log-likelihood (monotonic transform – same maximizer): $\ell(\theta) = \log L(\theta) = n_H \times \log(\theta) + n_T \times \log(1-\theta)$
Differentiate and set to zero: $d\ell/d\theta = n_H/\theta - n_T/(1-\theta) = 0$ $n_H \times (1-\theta) = n_T \times \theta$ $n_H - n_H \times \theta = n_T \times \theta$ $n_H = (n_H + n_T) \times \theta$ PROOF RESULT: $\theta_{MLE} = n_H / (n_H + n_T)$ QED
[fraction of heads observed in training data]

MLE Numerical Example: Observe: 7 Heads, 3 Tails in 10 flips $\theta_{MLE} = 7/10 = 0.70 \rightarrow$ estimate $P(\text{Head}) = 0.70$ Small sample problem: Observe: 3 Heads, 0 Tails in 3 flips $\theta_{MLE} = 3/3 = 1.0 \rightarrow$ says coin ALWAYS lands heads (clearly wrong!) Solution: MAP with a prior solves this problem.

(b) Maximum A Posteriori (MAP) Probability Algorithm:

Definition: MAP finds the hypothesis that maximizes the posterior probability $P(h|D)$ — combining the likelihood of data AND the prior belief about the hypothesis.

$h_{MAP} = \text{argmax}_h P(h | D) = \text{argmax}_h P(D|h) \times P(h) / P(D)$ [by Bayes theorem] = $\text{argmax}_h P(D|h) \times P(h)$ [$P(D)$ is constant w.r.t. h] Log form: $h_{MAP} = \text{argmax}_h [\sum_i \log P(d_i|h) + \log P(h)]$ ^^^^^^^ This term is absent in MLE

PROOF — MAP for Coin Flipping with Beta Prior:

Prior: $P(\theta) \propto \theta^{\alpha-1} \times (1-\theta)^{\beta-1}$ [Beta(α, β) distribution] Posterior: $P(\theta|D) \propto P(D|\theta) \times P(\theta) = \theta^{n_H} \times (1-\theta)^{n_T} \times \theta^{\alpha-1} \times (1-\theta)^{\beta-1} = \theta^{(n_H+\alpha-1)} \times (1-\theta)^{(n_T+\beta-1)}$ [This is a Beta($n_H+\alpha, n_T+\beta$) distribution] Mode of Beta(a, b) = $(a-1)/(a+b-2)$, so: $\theta_{MAP} = (n_H+\alpha-1) / (n_H+n_T+\alpha+\beta-2)$ Special case – Uniform prior ($\alpha=\beta=1$): $\theta_{MAP} = n_H/(n_H+n_T) = \theta_{MLE}$ [MAP equals MLE with uniform prior]

MAP Numerical Comparison: Data: 3 Heads, 0 Tails (small sample) Prior: Beta(2,2) — weak belief in fair coin ($\alpha=2, \beta=2$) MLE: $\theta_{MLE} = 3/3 = 1.00$ (extreme, unrealistic) MAP: $\theta_{MAP} = (3+2-1)/(3+0+2+2-2) = 4/5 = 0.80$ (more reasonable) With large data (300 Heads, 100 Tails): MLE: $300/400 = 0.750$ MAP: $(300+2-1)/(400+2+2-2) = 301/402 \approx 0.749$ (nearly identical) → With large data, MLE and MAP converge (data dominates prior) Key insight: MAP = MLE + regularization term $\log P(h)$ L2 regularization $\equiv$ MAP with Gaussian prior on weights L1 regularization $\equiv$ MAP with Laplace prior on weights

Answer

Q10. Give Decision Trees for: (i) $A \wedge \neg B$ (ii) $A \vee [B \wedge C]$

A **Decision Tree** represents a Boolean function as a hierarchical set of attribute tests. Each internal node tests an attribute, branches correspond to attribute values, and leaves specify the output (True/False).

(i) $A \wedge \neg B$ (A AND NOT B):

This function is TRUE only when A=True AND B=False simultaneously.

Decision Tree for A AND NOT-B: [A] / \ A=No A=Yes | | FALSE [B] / \ B=Yes B=No | | FALSE TRUE Truth Table Verification: A=F, B=T: A=No $\rightarrow$ FALSE ✓ ($\neg$ FALSE = T, T AND F = F) A=F, B=F: A=No $\rightarrow$ FALSE ✓ (False AND True = False) A=T, B=T: A=Yes, B=Yes $\rightarrow$ FALSE ✓ (True AND False = False) A=T, B=F: A=Yes, B=No $\rightarrow$ TRUE ✓ (True AND True = True) Logic: First test A (if False $\rightarrow$ immediately False since AND needs both True). If A=True, then test B (if B=True $\rightarrow$ False because we need NOT B).

(ii) $A \vee [B \wedge C]$ (A OR (B AND C)):

This function is TRUE when: A=True, OR when both B=True AND C=True. It is FALSE when A=False AND (B=False OR C=False).

Decision Tree for A OR (B AND C): [A] / \ A=Yes A=No | | TRUE [B] / \ B=No B=Yes | | FALSE [C] / \ C=No C=Yes | | FALSE TRUE Truth Table Verification (key cases): A=T, B=any, C=any: A=Yes $\rightarrow$ TRUE ✓ (A is True $\rightarrow$ whole OR is True) A=F, B=F, C=any: A=No, B=No $\rightarrow$ FALSE ✓ (B needed for AND) A=F, B=T, C=F: A=No, B=Yes, C=No $\rightarrow$ FALSE ✓ (C needed for AND) A=F, B=T, C=T: A=No, B=Yes, C=Yes $\rightarrow$ TRUE ✓ (B AND C is True) Logic: Test A first (if True $\rightarrow$ short-circuit OR $\rightarrow$ True immediately). If A=False, must check B AND C: test B, then test C. Only return True if both B=True AND C=True.

Additional Decision Tree concept — ID3 Algorithm:

The ID3 algorithm builds decision trees using Information Gain to choose the best attribute at each node:

- At each node, compute IG for every remaining attribute
- Select attribute with HIGHEST IG as the split attribute
- Create branches for each value of that attribute
- Recurse on each branch with remaining examples and attributes
- Stop when: all examples in a branch have same class, or no more attributes

$IG(S, A) = H(S) - \sum_v [|S_v|/|S| \times H(S_v)]$ $H(S) = -\sum_c [p(c) \times \log_2(p(c))]$ [Shannon entropy] Rule: Choose $A^* = \text{argmax}_A IG(S, A)$ at each node

Unit — Quick Reference — Key Formulas & Definitions

Formula / Concept	Expression
Entropy	$H(S) = -\sum [p(i) \cdot \log_2(p(i))]$
Information Gain	$IG(S,A) = H(S) - \sum_v [(S_v / S) \cdot H(S_v)]$
Bayes Theorem	$P(h D) = P(D h) \cdot P(h) / P(D)$
MLE	$h_{MLE} = \operatorname{argmax}_h P(D h) = \operatorname{argmax}_h \sum_i \log P(d_i h)$
MAP	$h_{MAP} = \operatorname{argmax}_h P(D h) \cdot P(h) = \operatorname{argmax}_h [\sum \log P(d_i h) + \log P(h)]$
Coin MLE	$\theta_{MLE} = n_{heads} / (n_{heads} + n_{tails})$
Coin MAP (Beta prior)	$\theta_{MAP} = (n_H + \alpha - 1) / (n_H + n_T + \alpha + \beta - 2)$
Version Space	$VS(H,D) = \{h \in H \mid h(x)=c(x) \text{ for all } (x,c(x)) \text{ in } D\}$
VS Representation Thm	$h \text{ in VS iff exists } s \text{ in } S, g \text{ in } G: g \geq_g h \geq_g s$
General-to-Specific	$h_j \geq_g h_i \text{ iff } \{x:h_i(x)=1\} \subseteq \{x:h_j(x)=1\}$
FIND-S Update	if $h[i]=\text{empty}$: $h[i]=x_i$; elif $h[i] \neq x_i$: $h[i]=?$
ID3 Attribute Selection	$A^* = \operatorname{argmax}_A IG(S,A)$
A AND NOT-B	TRUE only when $A=\text{True}$ AND $B=\text{False}$
A OR (B AND C)	TRUE when $A=\text{True}$, OR when $(B=\text{True} \text{ AND } C=\text{True})$